

北京邮电大学

实验报告



题目： 拆解二进制炸弹

班 级： _____

学 号： _____

姓 名： _____

学 院： _____

年 月 日

一、实验目的

1. 理解 C 语言程序的机器级表示。
2. 初步掌握 GDB 调试器的用法。
3. 阅读 C 编译器生成的**鲲鹏（arm）** 机器代码，理解不同控制结构生成的基本指令模式，过程的实现。

二、实验环境

1. 服务器
服务器：10.99.0.230（arm 版本）：**2025 年 12 月 14 日 23:59 分关机；提交作业截止时间 2025 年 12 月 18 日 23:59，延迟扣分（每晚一天扣 1.5 分）**
2. MobaXterm
3. Linux
4. Objdump 命令反汇编
5. GDB 调试工具
6. 积分榜（<http://10.99.0.238:19210/scoreboard>）

三、实验内容

登录 bupt1 服务器，在 home 目录下可以找到 Evil 博士专门为你量身定制的一个 bomb，当运行时，它会要求你输入一个字符串，如果正确，则进入下一关，继续要求你输入下一个字符串；否则，炸弹就会爆炸，输出一行提示信息并向计分服务器提交扣分信息。因此，本实验要求你必须通过反汇编和逆向工程对 bomb 执行文件进行分析，找到正确的字符串来解除这个的炸弹。

本实验通过要求使用课程所学知识拆除一个“binary bombs”来增强对程序的机器级表示、汇编语言、调试器和逆向工程等方面原理与技能的掌握。“binary bombs”是一个 Linux 可执行程序，包含了 5 个阶段（或关卡）。炸弹运行的每个阶段要求你输入一个特定字符串，你的输入符合程序预期的输入，该阶段的炸弹就被拆除引信；否则炸弹“爆炸”，打印输出“BOOM!!!”。炸弹的每个阶段考察了机器级程序语言的一个不同方面，难度逐级递增。

为完成二进制炸弹拆除任务，需要使用 gdb 调试器和 objdump 来反汇编 bomb 文件，可以单步跟踪调试每一阶段的机器代码，也可以阅读反汇编代码，从中理解每一汇编语言代码的行为或作用，进而设法推断拆除炸弹所需的目标字符串。实验 2 的具体内容见实验 2 说明。

四、实验步骤及实验分析

准备工作：

在拆除了 LINUX 服务器上的炸弹之后，对于鲲鹏上的炸弹其实也是异曲同工，但是要做到从 X86 到 ARM64 的转变，虽然逻辑方法一样，首先还是得先阅读指令集和入门指南，先了解在新的架构上，我跟机器沟通的语言是什么，以及其寄存器内存的改变，以下是我的初步梳理：

● 寄存器的规则

- ① 通用寄存器命名：在 X86 中，寄存器为 %rax, %rbx 之类的，而在 ARM64 中，通用寄存器有 31 个，命名较为简单，为 X0-X30，这为我们阅读代码提供了更方便的阅读体验。
- ② 访问规则：在访问时，X86 是使用 q, b 等后缀指定访问位数，而 ARM64 中使用 Xn 表示 64 位，Wn 表示低 32 位等。
- ③ 通用寄存器规定：在 X86 中我们知道了 %rax 是用来存取返回值，%rdx, %rdi 等是用来存取参数和用

户输入。而在 ARM64 中，一般来说 X0-X7 用于传递参数，即拆弹的时候输入的 input_strings 是存放在 X0, X1 等中的；X0 也用于存放函数返回值，在之前 X86 中类似 string_not_equal 之类的函数返回值会存在这之中；X29 是帧指针，类似于 X86 的 %rbp；X30 是链接寄存器，是跟 X86 区别的地方，在 ARM64 中，call 指令不会自动压栈返回地址，而是存入 X30，函数 ret 时跳转 X30。

④ 一些特殊寄存器的规则：SP 指的是堆栈指针，XZR/WZR 时零寄存器，永远读取为 0，写入数据无效；PC 是程序计数器，指向当前指令。

● 内存访问：

在 X86 中可以直接在内存之间做运算，要先把数据拷贝到寄存器，等运算结束再拷贝回去

● 栈操作：

stp: stp x29, x30, [sp, #-32]! ; 将帧指针和链接寄存器 LR 存入栈中，将 SP 先减去 32 字节，然后将数据存入新的栈顶（函数序言的典型操作）

mov: mov X29, sp; 将 SP 复制给 FP，建立新的栈帧

Ldp: ldp X29, X30, [SP], #32,, 表示从栈中恢复 X29 和 X30 的值（函数尾声）

Ret: 返回到调用者，跳转

● 控制流：

比较：CMP，更新标志位

跳转：B.EQ, B.NE 等

比较并跳转：CBZ A, B, 表示若 A 是 0，则跳转到 B，指令如 CBZ, CBNE 等

● 运算指令：

加减法：ADD/SUB，后面加的后缀 S 等表示更新标志位

乘法：MUL/MADD，如 MADD Xd, Xn, Xm, Xa 计算 $X_d = X_a + (X_n * X_m)$

位操作：AND, ORR, EOR, LSL（左移）, LSR（右移）

计算地址：ADRP

然后开始入手代码，先反汇编得到 main 函数的汇编代码

```
(gdb) disas main
Dump of assembler code for function main:
0x00000000400ef4 <+0>:      stp     x29, x30, [sp, #-32]!
0x00000000400ef8 <+4>:      mov     x29, sp
0x00000000400efc <+8>:      cmp     w0, #0x1
0x00000000400f00 <+12>:     b.eq     0x400fe0 <main+236> // b.none
0x00000000400f04 <+16>:     str     x19, [x29, #16]
0x00000000400f08 <+20>:     mov     x19, x1
0x00000000400f0c <+24>:     cmp     w0, #0x2
0x00000000400f10 <+28>:     b.ne     0x401010 <main+284> // b.any
0x00000000400f14 <+32>:     adrp    x1, 0x402000 <submitr+896>
0x00000000400f18 <+36>:     add     x1, x1, #0x580
0x00000000400f1c <+40>:     ldr     x0, [x19, #8]
0x00000000400f20 <+44>:     bl      0x400c40 <fopen@plt>
0x00000000400f24 <+48>:     adrp    x1, 0x420000 <strlen@got.plt>
0x00000000400f28 <+52>:     str     x0, [x1, #1824]
0x00000000400f2c <+56>:     cbz     x0, 0x400ff4 <main+256>
0x00000000400f30 <+60>:     ldr     x19, [x29, #16]
0x00000000400f34 <+64>:     bl      0x4016b0 <initialize_bomb>
0x00000000400f38 <+68>:     adrp    x0, 0x402000 <submitr+896>
0x00000000400f3c <+72>:     add     x0, x0, #0x5c8
0x00000000400f40 <+76>:     bl      0x400cd0 <puts@plt>
0x00000000400f44 <+80>:     adrp    x0, 0x402000 <submitr+896>
0x00000000400f48 <+84>:     add     x0, x0, #0x608
0x00000000400f4c <+88>:     bl      0x400cd0 <puts@plt>
0x00000000400f50 <+92>:     bl      0x401998 <read_line>
0x00000000400f54 <+96>:     bl      0x401028 <phase_1>
0x00000000400f58 <+100>:    bl      0x401ac4 <phase_defused>
0x00000000400f5c <+104>:    adrp    x0, 0x402000 <submitr+896>
0x00000000400f60 <+108>:    add     x0, x0, #0x638
--Type <RET> for more, q to quit, c to continue without paging--ret
```



```

0x0000000000400f64 <+112>: bl      0x400cd0 <puts@plt>
0x0000000000400f68 <+116>: bl      0x401998 <read_line>
0x0000000000400f6c <+120>: bl      0x401050 <phase_2>
0x0000000000400f70 <+124>: bl      0x401ac4 <phase_defused>
0x0000000000400f74 <+128>: adrp    x0, 0x402000 <submitr+896>
0x0000000000400f78 <+132>: add     x0, x0, #0x668
0x0000000000400f7c <+136>: bl      0x400cd0 <puts@plt>
0x0000000000400f80 <+140>: bl      0x401998 <read_line>
0x0000000000400f84 <+144>: bl      0x4010c4 <phase_3>
0x0000000000400f88 <+148>: bl      0x401ac4 <phase_defused>
0x0000000000400f8c <+152>: adrp    x0, 0x402000 <submitr+896>
0x0000000000400f90 <+156>: add     x0, x0, #0x688
0x0000000000400f94 <+160>: bl      0x400cd0 <puts@plt>
0x0000000000400f98 <+164>: bl      0x401998 <read_line>
0x0000000000400f9c <+168>: bl      0x401284 <phase_4>
0x0000000000400fa0 <+172>: bl      0x401ac4 <phase_defused>
0x0000000000400fa4 <+176>: adrp    x0, 0x402000 <submitr+896>
0x0000000000400fa8 <+180>: add     x0, x0, #0x698
0x0000000000400fac <+184>: bl      0x400cd0 <puts@plt>
0x0000000000400fb0 <+188>: bl      0x401998 <read_line>
0x0000000000400fb4 <+192>: bl      0x4012e8 <phase_5>
0x0000000000400fb8 <+196>: bl      0x401ac4 <phase_defused>
0x0000000000400fbc <+200>: adrp    x0, 0x402000 <submitr+896>
0x0000000000400fc0 <+204>: add     x0, x0, #0x6c0
0x0000000000400fc4 <+208>: bl      0x400cd0 <puts@plt>
0x0000000000400fc8 <+212>: bl      0x401998 <read_line>
0x0000000000400fcc <+216>: bl      0x401364 <phase_6>
0x0000000000400fd0 <+220>: bl      0x401ac4 <phase_defused>
0x0000000000400fd4 <+224>: mov     w0, #0x0 // #0
0x0000000000400fd8 <+228>: ldp     x29, x30, [sp], #32
0x0000000000400fdc <+232>: ret
0x0000000000400fe0 <+236>: adrp    x0, 0x420000 <strlen@got.plt>
0x0000000000400fe4 <+240>: ldr     x1, [x0, #1808]
0x0000000000400fe8 <+244>: adrp    x0, 0x420000 <strlen@got.plt>
0x0000000000400fec <+248>: str     x1, [x0, #1824]
--Type <RET> for more, q to quit, c to continue without paging--ret

```

```

--Type <RET> for more, q to quit, c to continue without paging--ret
0x0000000000400ff0 <+252>: b       0x400f34 <main+64>
0x0000000000400ff4 <+256>: ldr     x2, [x19, #8]
0x0000000000400ff8 <+260>: ldr     x1, [x19]
0x0000000000400ffc <+264>: adrp    x0, 0x402000 <submitr+896>
0x0000000000401000 <+268>: add     x0, x0, #0x588
0x0000000000401004 <+272>: bl      0x400d80 <printf@plt>
0x0000000000401008 <+276>: mov     w0, #0x8 // #8
0x000000000040100c <+280>: bl      0x400c10 <exit@plt>
0x0000000000401010 <+284>: ldr     x1, [x1]
0x0000000000401014 <+288>: adrp    x0, 0x402000 <submitr+896>
0x0000000000401018 <+292>: add     x0, x0, #0x5a8
0x000000000040101c <+296>: bl      0x400d80 <printf@plt>
0x0000000000401020 <+300>: mov     w0, #0x8 // #8
0x0000000000401024 <+304>: bl      0x400c10 <exit@plt>
End of assembler dump.

```

得到此次也是拆除六个显式炸弹，然后跟 X86 一样，反汇编拆除炸弹函数得到还有一个隐藏炸弹：

```

0x401b38 <phase_defused+116> add     x0, x29, #0x20
0x401b3c <phase_defused+120> bl      0x401618 <strings_not_equal>
0x401b40 <phase_defused+124> cbnz    w0, 0x401b14 <phase_defused+80>
0x401b44 <phase_defused+128> adrp    x0, 0x402000 <submitr+896>
0x401b48 <phase_defused+132> add     x0, x0, #0x988
0x401b4c <phase_defused+136> bl      0x400cd0 <puts@plt>
0x401b50 <phase_defused+140> adrp    x0, 0x402000 <submitr+896>
0x401b54 <phase_defused+144> add     x0, x0, #0x9b0
0x401b58 <phase_defused+148> bl      0x400cd0 <puts@plt>
0x401b5c <phase_defused+152> bl      0x40150c <secret_phase>
0x401b60 <phase_defused+156> b       0x401b14 <phase_defused+80>
0x401b64 <sigalarm_handler> stp     x29, x30, [sp, #-16]!
0x401b68 <sigalarm_handler+4> mov     x29, sp
0x401b6c <sigalarm_handler+8> mov     w2, #0x0 // #0
0x401b70 <sigalarm_handler+12> adrp    x1, 0x402000 <submitr+896>
0x401b74 <sigalarm_handler+16> add     x1, x1, #0xaa0
0x401b78 <sigalarm_handler+20> adrp    x0, 0x420000 <strlen@got.plt>
0x401b7c <sigalarm_handler+24> ldr     x0, [x0, #1792]

```

阶段 1—拆除第一个炸弹：

进入 gdb，设置断点，反汇编 phase_1，得到汇编代码：

```
(gdb) b explode_bomb
Breakpoint 1 at 0x40192c
(gdb) b phase_2
Breakpoint 2 at 0x401064
(gdb) disas phase_1
Dump of assembler code for function phase_1:
   0x0000000000401028 <+0>:      stp     x29, x30, [sp, #-16]!
   0x000000000040102c <+4>:      mov     x29, sp
   0x0000000000401030 <+8>:      adrp    x1, 0x402000 <submitr+896>
   0x0000000000401034 <+12>:     add     x1, x1, #0x6f0
   0x0000000000401038 <+16>:     bl      0x401618 <strings_not_equal>
   0x000000000040103c <+20>:     cbnz    w0, 0x401048 <phase_1+32>
   0x0000000000401040 <+24>:     ldp     x29, x30, [sp], #16
   0x0000000000401044 <+28>:     ret
   0x0000000000401048 <+32>:     bl      0x40191c <explode_bomb>
   0x000000000040104c <+36>:     b       0x401040 <phase_1+24>
End of assembler dump.
```

这是一段比较简单的代码，阅读汇编代码，可以得出，在函数开头是一个简单的算地址指令，得到的地址再与我们输入的字符串作比较，所以我们根据它的计算方法： $0x402000 + 0x6f0$ 得到密钥的地址为 $0x4026f0$ ，此时将断点设置在函数入口，打印地址内容得到密钥

```
(gdb) b phase_1
Breakpoint 3 at 0x401038
(gdb) run
Starting program: /students/2024211139/bomb102/bomb
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
1

Breakpoint 3, 0x0000000000401038 in phase_1 ()
(gdb) x/s 0x4026f0
0x4026f0:      "When I get angry, Mr. Bigglesworth gets upset."
```

此时可以得到密钥为：

When I get angry, Mr. Bigglesworth gets upset.

同样，创建一个密钥文件，方便以后运行测试，将密钥存入 answer.txt

阶段 2—拆除第二个炸弹：

```
(gdb) disas phase_2
Dump of assembler code for function phase_2:
0x0000000000401050 <+0>:      stp     x29, x30, [sp, #-80]!
0x0000000000401054 <+4>:      mov     x29, sp
0x0000000000401058 <+8>:      stp     x19, x20, [sp, #16]
0x000000000040105c <+12>:     str     x21, [sp, #32]
0x0000000000401060 <+16>:     add     x1, x29, #0x38
=> 0x0000000000401064 <+20>:     bl      0x401958 <read_six_numbers>
0x0000000000401068 <+24>:     ldr     w0, [x29, #56]
0x000000000040106c <+28>:     tbnz    w0, #31, 0x401080 <phase_2+48>
0x0000000000401070 <+32>:     mov     x20, #0x0                                // #0
0x0000000000401074 <+36>:     mov     x19, #0x1                                // #1
0x0000000000401078 <+40>:     add     x21, x29, #0x38
0x000000000040107c <+44>:     b       0x401098 <phase_2+72>
0x0000000000401080 <+48>:     bl      0x40191c <explode_bomb>
0x0000000000401084 <+52>:     b       0x401070 <phase_2+32>
0x0000000000401088 <+56>:     add     x19, x19, #0x1
0x000000000040108c <+60>:     add     x20, x20, #0x4
0x0000000000401090 <+64>:     cmp     x19, #0x6
0x0000000000401094 <+68>:     b.eq    0x4010b4 <phase_2+100> // b.none
0x0000000000401098 <+72>:     ldr     w0, [x20, x21]
0x000000000040109c <+76>:     add     w0, w0, w19
0x00000000004010a0 <+80>:     ldr     w1, [x21, x19, lsl #2]
0x00000000004010a4 <+84>:     cmp     w1, w0
0x00000000004010a8 <+88>:     b.eq    0x401088 <phase_2+56> // b.none
0x00000000004010ac <+92>:     bl      0x40191c <explode_bomb>
0x00000000004010b0 <+96>:     b       0x401088 <phase_2+56>
0x00000000004010b4 <+100>:    ldp     x19, x20, [sp, #16]
0x00000000004010b8 <+104>:    ldr     x21, [sp, #32]
0x00000000004010bc <+108>:    ldp     x29, x30, [sp], #80
0x00000000004010c0 <+112>:    ret
End of assembler dump.
```

反汇编 phase_2 得到汇编代码，阅读分析汇编代码，这是一个迭代的函数，要求我们输入六个数字，要求第一个数字为非负数，且之后的数字满足迭代关系为： $N_{i+1} = N_i + i$ 定第一个数为 1，则这个序列为 1 2 4 7 11 16，存入密钥

阶段 3—拆除第三个炸弹：

```
Dump of assembler code for function phase_3:
0x0000000004010c4 <+0>:      stp     x29, x30, [sp, #-32]!
0x0000000004010c8 <+4>:      mov     x29, sp
0x0000000004010cc <+8>:      add     x4, x29, #0x18
0x0000000004010d0 <+12>:     add     x3, x29, #0x17
0x0000000004010d4 <+16>:     add     x2, x29, #0x1c
0x0000000004010d8 <+20>:     adrp    x1, 0x402000 <submitr+896>
0x0000000004010dc <+24>:     add     x1, x1, #0x720
=> 0x0000000004010e0 <+28>:     bl      0x400d70 <__isoc99_sscanf@plt>
0x0000000004010e4 <+32>:     cmp     w0, #0x2
0x0000000004010e8 <+36>:     b.le    0x401134 <phase_3+112>
0x0000000004010ec <+40>:     ldr     w0, [x29, #28]
0x0000000004010f0 <+44>:     cmp     w0, #0x3
0x0000000004010f4 <+48>:     b.eq    0x4011a0 <phase_3+220> // b.none
0x0000000004010f8 <+52>:     b.le    0x40113c <phase_3+120>
0x0000000004010fc <+56>:     cmp     w0, #0x5
0x000000000401100 <+60>:     b.eq    0x4011d8 <phase_3+276> // b.none
0x000000000401104 <+64>:     b.lt    0x4011bc <phase_3+248> // b.tstop
0x000000000401108 <+68>:     cmp     w0, #0x6
0x00000000040110c <+72>:     b.eq    0x4011f4 <phase_3+304> // b.none
0x000000000401110 <+76>:     cmp     w0, #0x7
0x000000000401114 <+80>:     b.ne    0x401210 <phase_3+332> // b.any
0x000000000401118 <+84>:     mov     w0, #0x62 // #98
0x00000000040111c <+88>:     ldr     w1, [x29, #24]
0x000000000401120 <+92>:     cmp     w1, #0x50
0x000000000401124 <+96>:     b.eq    0x401218 <phase_3+340> // b.none
0x000000000401128 <+100>:    bl      0x40191c <explode_bomb>
0x00000000040112c <+104>:    mov     w0, #0x62 // #98
0x000000000401130 <+108>:    b      0x401218 <phase_3+340>
0x000000000401134 <+112>:    bl      0x40191c <explode_bomb>
0x000000000401138 <+116>:    b      0x4010ec <phase_3+40>
0x00000000040113c <+120>:    cmp     w0, #0x1
0x000000000401140 <+124>:    b.eq    0x401168 <phase_3+164> // b.none
0x000000000401144 <+128>:    b.gt    0x401184 <phase_3+192>
0x000000000401148 <+132>:    cbnz    w0, 0x401210 <phase_3+332>
0x00000000040114c <+136>:    mov     w0, #0x77 // #119
0x000000000401150 <+140>:    ldr     w1, [x29, #24]
0x000000000401154 <+144>:    cmp     w1, #0x9e
0x000000000401158 <+148>:    b.eq    0x401218 <phase_3+340> // b.none
0x00000000040115c <+152>:    bl      0x40191c <explode_bomb>
0x000000000401160 <+156>:    mov     w0, #0x77 // #119
0x000000000401164 <+160>:    b      0x401218 <phase_3+340>
0x000000000401168 <+164>:    mov     w0, #0x61 // #97
0x00000000040116c <+168>:    ldr     w1, [x29, #24]
0x000000000401170 <+172>:    cmp     w1, #0xcc
0x000000000401174 <+176>:    b.eq    0x401218 <phase_3+340> // b.none
0x000000000401178 <+180>:    bl      0x40191c <explode_bomb>
0x00000000040117c <+184>:    mov     w0, #0x61 // #97
0x000000000401180 <+188>:    b      0x401218 <phase_3+340>
0x000000000401184 <+192>:    mov     w0, #0x6f // #111
0x000000000401188 <+196>:    ldr     w1, [x29, #24]
0x00000000040118c <+200>:    cmp     w1, #0x274
0x000000000401190 <+204>:    b.eq    0x401218 <phase_3+340> // b.none
0x000000000401194 <+208>:    bl      0x40191c <explode_bomb>
0x000000000401198 <+212>:    mov     w0, #0x6f // #111
0x00000000040119c <+216>:    b      0x401218 <phase_3+340>
0x0000000004011a0 <+220>:    mov     w0, #0x69 // #105
--Type <RET> for more, q to quit, c to continue without paging--ret
```

```

--Type <RET> for more, q to quit, c to continue without paging--ret
0x00000000004011a4 <+224>: ldr    w1, [x29, #24]
0x00000000004011a8 <+228>: cmp    w1, #0xc1
0x00000000004011ac <+232>: b.eq   0x401218 <phase_3+340> // b.none
0x00000000004011b0 <+236>: bl     0x40191c <explode_bomb>
0x00000000004011b4 <+240>: mov    w0, #0x69 // #105
0x00000000004011b8 <+244>: b      0x401218 <phase_3+340>
0x00000000004011bc <+248>: mov    w0, #0x6a // #106
0x00000000004011c0 <+252>: ldr    w1, [x29, #24]
0x00000000004011c4 <+256>: cmp    w1, #0x37b
0x00000000004011c8 <+260>: b.eq   0x401218 <phase_3+340> // b.none
0x00000000004011cc <+264>: bl     0x40191c <explode_bomb>
0x00000000004011d0 <+268>: mov    w0, #0x6a // #106
0x00000000004011d4 <+272>: b      0x401218 <phase_3+340>
0x00000000004011d8 <+276>: mov    w0, #0x6c // #108
0x00000000004011dc <+280>: ldr    w1, [x29, #24]
0x00000000004011e0 <+284>: cmp    w1, #0xfbf
0x00000000004011e4 <+288>: b.eq   0x401218 <phase_3+340> // b.none
0x00000000004011e8 <+292>: bl     0x40191c <explode_bomb>
0x00000000004011ec <+296>: mov    w0, #0x6c // #108
0x00000000004011f0 <+300>: b      0x401218 <phase_3+340>
0x00000000004011f4 <+304>: mov    w0, #0x78 // #120
0x00000000004011f8 <+308>: ldr    w1, [x29, #24]
0x00000000004011fc <+312>: cmp    w1, #0x2af
0x0000000000401200 <+316>: b.eq   0x401218 <phase_3+340> // b.none
0x0000000000401204 <+320>: bl     0x40191c <explode_bomb>
0x0000000000401208 <+324>: mov    w0, #0x78 // #120
0x000000000040120c <+328>: b      0x401218 <phase_3+340>
0x0000000000401210 <+332>: bl     0x40191c <explode_bomb>
0x0000000000401214 <+336>: mov    w0, #0x6a // #106
0x0000000000401218 <+340>: ldrb   w1, [x29, #23]
0x000000000040121c <+344>: cmp    w1, w0
0x0000000000401220 <+348>: b.eq   0x401228 <phase_3+356> // b.none
0x0000000000401224 <+352>: bl     0x40191c <explode_bomb>
0x0000000000401228 <+356>: ldp    x29, x30, [sp], #32
0x000000000040122c <+360>: ret
End of assembler dump.

```

第三个炸弹是关于 switch 语句的判断，输入要求包含三个内容，第一个是 casek，k 只能处于 0~7 这八个数中的一个数，第二个输入是一个 char 型字符，在汇编中给出了对应的 ASCII 码值，第三个输入也是一个数字，这个输入也在每个 case 的汇编代码中给出，整合代码块得到下面的 switch 跳转的期望表

Casek	汇编地址	字符 ASCII 码	数字期望值
0	0x40114c	0x77(w)	158
1	0x401168	0x61(a)	204
2	0x401184	0x6f(o)	628
3	0x4011a0	0x69(i)	193
4	0x4011bc	0x6a(j)	891
5	0x4011d8	0x6c(l)	251
6	0x4011f4	0x78(x)	687
7	0x401118	0x62(b)	80

取 k=0 的一组：0 w 158 存入密钥文件 y

阶段 4—拆除第四个炸弹：

```
(gdb) disas phase_4
Dump of assembler code for function phase_4:
0x0000000000401284 <+0>:    stp     x29, x30, [sp, #-32]!
0x0000000000401288 <+4>:    mov     x29, sp
0x000000000040128c <+8>:    add     x3, x29, #0x18
0x0000000000401290 <+12>:   add     x2, x29, #0x1c
0x0000000000401294 <+16>:   adrp    x1, 0x402000 <submitr+896>
0x0000000000401298 <+20>:   add     x1, x1, #0x730
=> 0x000000000040129c <+24>:   bl      0x400d70 <__isoc99_sscanf@plt>
0x00000000004012a0 <+28>:   cmp     w0, #0x2
0x00000000004012a4 <+32>:   b.ne    0x4012b4 <phase_4+48> // b.any
0x00000000004012a8 <+36>:   ldr     w0, [x29, #28]
0x00000000004012ac <+40>:   cmp     w0, #0xe
0x00000000004012b0 <+44>:   b.lsl   0x4012b8 <phase_4+52> // b.plast
0x00000000004012b4 <+48>:   bl      0x40191c <explode_bomb>
0x00000000004012b8 <+52>:   mov     w2, #0xe // #14
0x00000000004012bc <+56>:   mov     w1, #0x0 // #0
0x00000000004012c0 <+60>:   ldr     w0, [x29, #28]
0x00000000004012c4 <+64>:   bl      0x401230 <func4>
0x00000000004012c8 <+68>:   cmp     w0, #0xa
0x00000000004012cc <+72>:   b.ne    0x4012dc <phase_4+88> // b.any
0x00000000004012d0 <+76>:   ldr     w0, [x29, #24]
0x00000000004012d4 <+80>:   cmp     w0, #0xa
0x00000000004012d8 <+84>:   b.eq    0x4012e0 <phase_4+92> // b.none
0x00000000004012dc <+88>:   bl      0x40191c <explode_bomb>
0x00000000004012e0 <+92>:   ldp     x29, x30, [sp], #32
0x00000000004012e4 <+96>:   ret
End of assembler dump.
```

```
(gdb) disas func4
Dump of assembler code for function func4:
0x0000000000401230 <+0>:    stp     x29, x30, [sp, #-32]!
0x0000000000401234 <+4>:    mov     x29, sp
0x0000000000401238 <+8>:    str     x19, [sp, #16]
0x000000000040123c <+12>:   sub     w19, w2, w1
0x0000000000401240 <+16>:   add     w19, w19, w19, lsr #31
0x0000000000401244 <+20>:   add     w19, w1, w19, asr #1
0x0000000000401248 <+24>:   cmp     w19, w0
0x000000000040124c <+28>:   b.gt    0x401264 <func4+52>
0x0000000000401250 <+32>:   b.lt    0x401274 <func4+68> // b.tstop
0x0000000000401254 <+36>:   mov     w0, w19
0x0000000000401258 <+40>:   ldr     x19, [sp, #16]
0x000000000040125c <+44>:   ldp     x29, x30, [sp], #32
0x0000000000401260 <+48>:   ret
0x0000000000401264 <+52>:   sub     w2, w19, #0x1
0x0000000000401268 <+56>:   bl      0x401230 <func4>
0x000000000040126c <+60>:   add     w19, w19, w0
0x0000000000401270 <+64>:   b       0x401254 <func4+36>
0x0000000000401274 <+68>:   add     w1, w19, #0x1
0x0000000000401278 <+72>:   bl      0x401230 <func4>
0x000000000040127c <+76>:   add     w19, w19, w0
0x0000000000401280 <+80>:   b       0x401254 <func4+36>
End of assembler dump.
```

第四个炸弹中途调用了函数，先看主程序逻辑，phase_4 规定了输入两个值，第一个值要求在 0~14 的

范围内，然后以第一个值 a 为参数调用了函数 func(a, 0, 14),检查函数的返回值是否为 10，并最终检查第二个输入的参数和返回值是否相同，说明第二个输入的值也必须为 10，那就是要看 func 的功能，a 为多少的时候返回值才为 10。

来到 func4 函数的汇编代码，它接收三个参数（设为 a,b,c），实现的是一个二分查找的过程，在给出的参数 b 和 c 作为最大值和最小值，每次用二分的方法去查找 a 在哪里，而这个函数返回值则是通过递归有：

$$\text{func}(a, \text{min}, \text{max}) = \begin{cases} \text{mid} & a = \text{mid} \\ \text{mid} + \text{func}(a, \text{min}, \text{mid}-1) & a < \text{mid} \\ \text{mid} + \text{func}(a, \text{mid}+1, \text{max}) & a > \text{mid} \end{cases}$$

通过 min = 0, max = 14 反推有：

第一次寻找：mid = 7，此时肯定要往更小的方向找，所以 min 为 0，max 更新为 6，mid 累加为 7

第二次寻找：mid = 3，此时 mid 累加已经为 10 符合题意，故此时 a = mid = 3

所以第一个参数 a 应该为 3，此题的密钥为：3 10

阶段 5—拆除第五个炸弹：

```
Dump of assembler code for function phase_5:
0x0000000004012e8 <+0>: stp x29, x30, [sp, #-48]!
0x0000000004012ec <+4>: mov x29, sp
0x0000000004012f0 <+8>: str x19, [sp, #16]
0x0000000004012f4 <+12>: mov x19, x0
=> 0x0000000004012f8 <+16>: bl 0x4015ec <string_length>
0x0000000004012fc <+20>: cmp w0, #0x6
0x000000000401300 <+24>: b.ne 0x401354 <phase_5+108> // b.any
0x000000000401304 <+28>: mov x0, #0x0 // #0
0x000000000401308 <+32>: add x3, x29, #0x28
0x00000000040130c <+36>: adrp x2, 0x402000 <submitr+896>
0x000000000401310 <+40>: add x2, x2, #0x6e0
0x000000000401314 <+44>: ldrb w1, [x19, x0]
0x000000000401318 <+48>: and w1, w1, #0xf
0x00000000040131c <+52>: ldrb w1, [x2, w1, sxtw]
0x000000000401320 <+56>: strb w1, [x0, x3]
0x000000000401324 <+60>: add x0, x0, #0x1
0x000000000401328 <+64>: cmp x0, #0x6
0x00000000040132c <+68>: b.ne 0x401314 <phase_5+44> // b.any
0x000000000401330 <+72>: strb wzr, [x29, #46]
0x000000000401334 <+76>: adrp x1, 0x402000 <submitr+896>
0x000000000401338 <+80>: add x1, x1, #0x738
0x00000000040133c <+84>: add x0, x29, #0x28
0x000000000401340 <+88>: bl 0x401618 <strings_not_equal>
0x000000000401344 <+92>: cbnz w0, 0x40135c <phase_5+116>
0x000000000401348 <+96>: ldr x19, [sp, #16]
0x00000000040134c <+100>: ldp x29, x30, [sp], #48
0x000000000401350 <+104>: ret
0x000000000401354 <+108>: bl 0x40191c <explode_bomb>
--Type <RET> for more, q to quit, c to continue without paging--ret
0x000000000401358 <+112>: b 0x401304 <phase_5+28>
0x00000000040135c <+116>: bl 0x40191c <explode_bomb>
0x000000000401360 <+120>: b 0x401348 <phase_5+96>
End of assembler dump.
```

这是一个对字符串的输入和比较问题，但是跟第一个普通的输入的字符串不一样，这是类似于每个字符有一个密钥，得拿到每一个字符的密钥才可以拼成一个完整的期望字符串，首先打印地址内容，得到期

```
(gdb) x/s 0x402738
0x402738: "sabres"
```

望字符串为：sabres

然后去打印查找表的内容（类似于查看密码本），汇编代码中显示一共 16 个字符：

```
(gdb) x/16cb 0x4026e0
0x4026e0 <array.4328>: 109 'm' 97 'a' 100 'd' 117 'u' 105 'i' 101 'e' 114 'r' 115 's'
0x4026e8 <array.4328+8>: 110 'n' 102 'f' 111 'o' 116 't' 118 'v' 98 'b' 121 'y' 108 'l'
```

这个查找表顺序序列为：

maduiersnftotvbyl

根据查找的规则：索引 = $i \& 0xF$ ，则可以得到每个 i 的值为：

（省去了 0x）37 31 4D 36 35 37

对应字符为：7 1 M 6 5 7

于是此炸弹的密钥为：71M657

阶段 6—拆除第六个炸弹：

```
Dump of assembler code for function phase_6:
0x000000000401364 <+0>: stp x29, x30, [sp, #-128]!
0x000000000401368 <+4>: mov x29, sp
0x00000000040136c <+8>: stp x19, x20, [sp, #16]
0x000000000401370 <+12>: str x21, [sp, #32]
0x000000000401374 <+16>: add x1, x29, #0x68
=> 0x000000000401378 <+20>: bl 0x401958 <read_six_numbers>
0x00000000040137c <+24>: mov x21, #0x0 // #0
0x000000000401380 <+28>: add x20, x29, #0x68
0x000000000401384 <+32>: b 0x4013b8 <phase_6+84>
0x000000000401388 <+36>: bl 0x40191c <explode_bomb>
0x00000000040138c <+40>: b 0x4013c8 <phase_6+100>
0x000000000401390 <+44>: add w19, w19, #0x1
0x000000000401394 <+48>: cmp w19, #0x5
0x000000000401398 <+52>: b.gt 0x4013b4 <phase_6+80>
0x00000000040139c <+56>: ldr w1, [x20, x21, lsl #2]
0x0000000004013a0 <+60>: ldr w0, [x20, w19, sxtw #2]
0x0000000004013a4 <+64>: cmp w1, w0
0x0000000004013a8 <+68>: b.ne 0x401390 <phase_6+44> // b.any
0x0000000004013ac <+72>: bl 0x40191c <explode_bomb>
0x0000000004013b0 <+76>: b 0x401390 <phase_6+44>
0x0000000004013b4 <+80>: add x21, x21, #0x1
0x0000000004013b8 <+84>: ldr w0, [x20, x21, lsl #2]
0x0000000004013bc <+88>: sub w0, w0, #0x1
0x0000000004013c0 <+92>: cmp w0, #0x5
0x0000000004013c4 <+96>: b.hi 0x401388 <phase_6+36> // b.pmore
0x0000000004013c8 <+100>: add w19, w21, #0x1
0x0000000004013cc <+104>: cmp x21, #0x5
0x0000000004013d0 <+108>: b.ne 0x40139c <phase_6+56> // b.any
0x0000000004013d4 <+112>: mov x0, #0x0 // #0
0x0000000004013d8 <+116>: add x2, x29, #0x68
0x0000000004013dc <+120>: mov w3, #0x7 // #7
0x0000000004013e0 <+124>: ldr w1, [x2, x0, lsl #2]
0x0000000004013e4 <+128>: sub w1, w3, w1
0x0000000004013e8 <+132>: str w1, [x2, x0, lsl #2]
0x0000000004013ec <+136>: add x0, x0, #0x1
0x0000000004013f0 <+140>: cmp x0, #0x6
0x0000000004013f4 <+144>: b.ne 0x4013e0 <phase_6+124> // b.any
0x0000000004013f8 <+148>: mov x3, #0x0 // #0
0x0000000004013fc <+152>: add x6, x29, #0x68
0x000000000401400 <+156>: adrp x4, 0x420000 <strlen@got.plt>
0x000000000401404 <+160>: add x5, x29, #0x38
0x000000000401408 <+164>: b 0x40142c <phase_6+200>
0x00000000040140c <+168>: ldr x1, [x1, #8]
0x000000000401410 <+172>: add w0, w0, #0x1
0x000000000401414 <+176>: cmp w0, w2
0x000000000401418 <+180>: b.ne 0x40140c <phase_6+168> // b.any
0x00000000040141c <+184>: str x1, [x5, x3, lsl #3]
0x000000000401420 <+188>: add x3, x3, #0x1
0x000000000401424 <+192>: cmp x3, #0x6
0x000000000401428 <+196>: b.eq 0x401444 <phase_6+224> // b.none
0x00000000040142c <+200>: ldr w2, [x6, x3, lsl #2]
0x000000000401430 <+204>: mov w0, #0x1 // #1
--Type <RET> for more, q to quit, c to continue without paging--ret
```



```

--Type <RET> for more, q to quit, c to continue without paging--ret
0x0000000000401434 <+208>: add    x1, x4, #0x110
0x0000000000401438 <+212>: cmp    w2, w0
0x000000000040143c <+216>: b.gt   0x40140c <phase_6+168>
0x0000000000401440 <+220>: b      0x40141c <phase_6+184>
0x0000000000401444 <+224>: ldr    x19, [x29, #56]
0x0000000000401448 <+228>: ldr    x0, [x29, #64]
0x000000000040144c <+232>: str    x0, [x19, #8]
0x0000000000401450 <+236>: ldr    x1, [x29, #72]
0x0000000000401454 <+240>: str    x1, [x0, #8]
0x0000000000401458 <+244>: ldr    x0, [x29, #80]
0x000000000040145c <+248>: str    x0, [x1, #8]
0x0000000000401460 <+252>: ldr    x1, [x29, #88]
0x0000000000401464 <+256>: str    x1, [x0, #8]
0x0000000000401468 <+260>: ldr    x0, [x29, #96]
0x000000000040146c <+264>: str    x0, [x1, #8]
0x0000000000401470 <+268>: str    xzr, [x0, #8]
0x0000000000401474 <+272>: mov    w20, #0x5 // #5
0x0000000000401478 <+276>: b      0x401488 <phase_6+292>
0x000000000040147c <+280>: ldr    x19, [x19, #8]
0x0000000000401480 <+284>: subs   w20, w20, #0x1
0x0000000000401484 <+288>: b.eq   0x4014a4 <phase_6+320> // b.none
0x0000000000401488 <+292>: ldr    x0, [x19, #8]
0x000000000040148c <+296>: ldr    w1, [x19]
0x0000000000401490 <+300>: ldr    w0, [x0]
0x0000000000401494 <+304>: cmp    w1, w0
0x0000000000401498 <+308>: b.ge   0x40147c <phase_6+280> // b.tcont
0x000000000040149c <+312>: bl     0x40191c <explode_bomb>
0x00000000004014a0 <+316>: b      0x40147c <phase_6+280>
0x00000000004014a4 <+320>: ldp    x19, x20, [sp, #16]
0x00000000004014a8 <+324>: ldr    x21, [sp, #32]
0x00000000004014ac <+328>: ldp    x29, x30, [sp], #128
0x00000000004014b0 <+332>: ret
End of assembler dump.

```

这是一个关于链表重组的问题，简单化就是：

对于一个初始链表有六个值 x1, x2...x6, 是乱序的，我们需要先知道六个值分别为多少，然后根据每个的值，把索引按照降序方式重新排列，然后经过 $N[i] = 7 - N[i]$ 的处理得到输入序列，我们需要输入的就是这个按照降序排列处理之后的索引序列，一个 1~6 的全排列，打印出原始链表的值：

```

(gdb) set $node = 0x420110
(gdb) printf "Node 1 Value: %d\n", *(int*)$node
Node 1 Value: 743
(gdb) set $node = *(long*)($node + 8)
(gdb) printf "Node Value: %d\n", *(int*)$node
Node Value: 313
(gdb) set $node = *(long*)($node + 8)
(gdb) printf "Node Value: %d\n", *(int*)$node
Node Value: 882
(gdb) set $node = *(long*)($node + 8)
(gdb) printf "Node Value: %d\n", *(int*)$node
Node Value: 464
(gdb) set $node = *(long*)($node + 8)
(gdb) printf "Node Value: %d\n", *(int*)$node
Node Value: 877
(gdb) set $node = *(long*)($node + 8)
(gdb) printf "Node Value: %d\n", *(int*)$node
Node Value: 71

```

得到原始排列为： 743 313 882 464 877 71

按照降序，其索引排列为： 3 5 1 4 2 6

处理之后的输入序列为： 4 2 6 3 5 1

阶段 7—拆除隐藏炸弹：

```
(gdb) disas phase_defused
Dump of assembler code for function phase_defused:
0x0000000000401ac4 <+0>: stp     x29, x30, [sp, #-112]!
0x0000000000401ac8 <+4>: mov     x29, sp
0x0000000000401acc <+8>: mov     w0, #0x1                                // #1
0x0000000000401ad0 <+12>: bl      0x40182c <send_msg>
0x0000000000401ad4 <+16>: adrp    x0, 0x420000 <strlen@got.plt>
0x0000000000401ad8 <+20>: ldr     w0, [x0, #1820]
0x0000000000401adc <+24>: cmp     w0, #0x6
0x0000000000401ae0 <+28>: b.eq    0x401aec <phase_defused+40> // b.none
0x0000000000401ae4 <+32>: ldp     x29, x30, [sp], #112
0x0000000000401ae8 <+36>: ret
0x0000000000401aec <+40>: add     x4, x29, #0x20
0x0000000000401af0 <+44>: add     x3, x29, #0x18
0x0000000000401af4 <+48>: add     x2, x29, #0x1c
0x0000000000401af8 <+52>: adrp    x1, 0x402000 <submitr+896>
0x0000000000401afc <+56>: add     x1, x1, #0x970
0x0000000000401b00 <+60>: adrp    x0, 0x420000 <strlen@got.plt>
0x0000000000401b04 <+64>: add     x0, x0, #0x818
0x0000000000401b08 <+68>: bl      0x400d70 <__isoc99_sscanf@plt>
0x0000000000401b0c <+72>: cmp     w0, #0x3
0x0000000000401b10 <+76>: b.eq    0x401b30 <phase_defused+108> // b.none
0x0000000000401b14 <+80>: adrp    x0, 0x402000 <submitr+896>
0x0000000000401b18 <+84>: add     x0, x0, #0x9e8
0x0000000000401b1c <+88>: bl      0x400cd0 <puts@plt>
0x0000000000401b20 <+92>: adrp    x0, 0x402000 <submitr+896>
0x0000000000401b24 <+96>: add     x0, x0, #0xa18
0x0000000000401b28 <+100>: bl      0x400cd0 <puts@plt>
0x0000000000401b2c <+104>: b       0x401ae4 <phase_defused+32>
0x0000000000401b30 <+108>: adrp    x1, 0x402000 <submitr+896>
0x0000000000401b34 <+112>: add     x1, x1, #0x980
0x0000000000401b38 <+116>: add     x0, x29, #0x20
0x0000000000401b3c <+120>: bl      0x401618 <strings_not_equal>
0x0000000000401b40 <+124>: cbnz    w0, 0x401b14 <phase_defused+80>
0x0000000000401b44 <+128>: adrp    x0, 0x402000 <submitr+896>
0x0000000000401b48 <+132>: add     x0, x0, #0x988
0x0000000000401b4c <+136>: bl      0x400cd0 <puts@plt>
0x0000000000401b50 <+140>: adrp    x0, 0x402000 <submitr+896>
0x0000000000401b54 <+144>: add     x0, x0, #0x9b0
0x0000000000401b58 <+148>: bl      0x400cd0 <puts@plt>
0x0000000000401b5c <+152>: bl      0x40150c <secret_phase>
0x0000000000401b60 <+156>: b       0x401b14 <phase_defused+80>
End of assembler dump.
```

阅读拆除炸弹的汇编代码，密钥同样是放在第四关之后的位置，打印密钥，为“DrEvil”，并未发生变化：

```
(gdb) x/s 0x402980
0x402980: "DrEvil"
```

加入密钥，进入隐藏关卡，反汇编得到汇编代码：

Dump of assembler code for function secret_phase:

```
0x000000000040150c <+0>: stp x29, x30, [sp, #-32]!
0x0000000000401510 <+4>: mov x29, sp
0x0000000000401514 <+8>: str x19, [sp, #16]
0x0000000000401518 <+12>: bl 0x401998 <read_line>
0x000000000040151c <+16>: mov w2, #0xa // #10
0x0000000000401520 <+20>: mov x1, #0x0 // #0
0x0000000000401524 <+24>: bl 0x400d00 <strtol@plt>
0x0000000000401528 <+28>: mov x19, x0
0x000000000040152c <+32>: sub w0, w0, #0x1
0x0000000000401530 <+36>: cmp w0, #0x3e8
0x0000000000401534 <+40>: b.hi 0x401574 <secret_phase+104> // b.pmore
0x0000000000401538 <+44>: mov w1, w19
0x000000000040153c <+48>: adrp x0, 0x420000 <strlen@got.plt>
0x0000000000401540 <+52>: add x0, x0, #0x110
0x0000000000401544 <+56>: add x0, x0, #0x10
0x0000000000401548 <+60>: bl 0x4014b4 <fun7>
0x000000000040154c <+64>: cmp w0, #0x3
0x0000000000401550 <+68>: b.eq 0x401558 <secret_phase+76> // b.none
0x0000000000401554 <+72>: bl 0x40191c <explode_bomb>
0x0000000000401558 <+76>: adrp x0, 0x402000 <submitr+896>
0x000000000040155c <+80>: add x0, x0, #0x740
0x0000000000401560 <+84>: bl 0x400cd0 <puts@plt>
0x0000000000401564 <+88>: bl 0x401ac4 <phase_defused>
0x0000000000401568 <+92>: ldr x19, [sp, #16]
0x000000000040156c <+96>: ldp x29, x30, [sp], #32
0x0000000000401570 <+100>: ret
0x0000000000401574 <+104>: bl 0x40191c <explode_bomb>
0x0000000000401578 <+108>: b 0x401538 <secret_phase+44>
```

End of assembler dump.

Dump of assembler code for function fun7:

```
0x00000000004014b4 <+0>: cbz x0, 0x401504 <fun7+80>
0x00000000004014b8 <+4>: stp x29, x30, [sp, #-16]!
0x00000000004014bc <+8>: mov x29, sp
0x00000000004014c0 <+12>: mov x2, x0
0x00000000004014c4 <+16>: ldr w3, [x0]
0x00000000004014c8 <+20>: cmp w3, w1
0x00000000004014cc <+24>: b.gt 0x4014e0 <fun7+44>
0x00000000004014d0 <+28>: mov w0, #0x0 // #0
0x00000000004014d4 <+32>: b.ne 0x4014f0 <fun7+60> // b.any
0x00000000004014d8 <+36>: ldp x29, x30, [sp], #16
0x00000000004014dc <+40>: ret
0x00000000004014e0 <+44>: ldr x0, [x0, #8]
0x00000000004014e4 <+48>: bl 0x4014b4 <fun7>
0x00000000004014e8 <+52>: lsl w0, w0, #1
0x00000000004014ec <+56>: b 0x4014d8 <fun7+36>
0x00000000004014f0 <+60>: ldr x0, [x2, #16]
0x00000000004014f4 <+64>: bl 0x4014b4 <fun7>
0x00000000004014f8 <+68>: lsl w0, w0, #1
0x00000000004014fc <+72>: add w0, w0, #0x1
0x0000000000401500 <+76>: b 0x4014d8 <fun7+36>
0x0000000000401504 <+80>: mov w0, #0xffffffff // #-1
0x0000000000401508 <+84>: ret
```

End of assembler dump.

得到了关于 secret_phase 和其中调用的函数 fun7 的汇编代码, 主函数中要求输入一个参数 N, 范围在 1~1001 之间, 然后将这个 N 作为 fun7 的第二个参数, fun7 的第一个参数为 0x420120, 是一个地址, 根据 X86 的实验经验, 这很有可能是一个二叉树的查找问题, 最终要求 fun7 的返回值必须为 3。

接下来分析 fun7 的函数块, fun7 是一个递归查找函数, 查找的目标就是第二个参数, 即我们输入的 N, 查找递归逻辑和 X86 的逻辑一模一样:

$$\text{Fun7}(n, x) = \begin{cases} -1 & , n == \text{null} \\ 0 & , n == x \\ 2 * \text{fun7}(n \rightarrow \text{first}, x), & x < n \rightarrow \text{val} \\ 2 * \text{fun7}(n \rightarrow \text{second}, x) + 1, & x > n \rightarrow \text{val} \end{cases}$$

返回值为 3，则是正好根->右->右（其实一开始可以多次左，但是为了方便，直接两次右即可）
 对树的节点进行打印，得到对应节点的值为：6b（107）

```

(gdb) x/4gx 0x420120
0x420120 <n1>: 0x0000000000000024      0x0000000000420138
0x420130 <n1+16>: 0x0000000000420150      0x0000000000000008
(gdb) x/4gx 0x0000000000420150
0x420150 <n22>: 0x0000000000000032      0x0000000000420180
0x420160 <n22+16>: 0x00000000004201b0      0x0000000000000016
(gdb) x/4gx 0x00000000004201b0
0x4201b0 <n34>: 0x000000000000006b      0x00000000004201f8
0x4201c0 <n34+16>: 0x0000000000420270      0x0000000000000028
  
```

所以最终密钥就为 107

五、总结体会

总结心得（包括实验过程中遇到的问题、如何解决的、过关或挫败的感受、实验投入的时间和精力、意见和建议等）